What survives when a paper leaves the lab: re-testing 22 robot-learning papers on a €1,350 rig

Ravindu Somawansa
Shisso Robotics
ravindu.somawansa@shissorobotics.com

Abstract. Of 37 claims taken from 22 robot-learning papers, 12 held when we re-tested them on an SO-101 arm driven by a Jetson Orin NX, a rig that costs about €1,350. Most of these papers were shown in simulation or on far more expensive hardware, and it remains unclear how much of what they report survives on the kind of setup a small team can afford. To this end, we keep a register of pre-registered re-tests: for every claim, the pass mark is committed before the run, and the cheapest baseline the method must beat (copy-last, the commanded joints or episode length) is evaluated on the same rows. We find that the failures fall into three classes, which we propose as a working taxonomy: methods that miss the real-time budget of the edge GPU, zero-shot judges beaten by a signal as trivial as episode length, and latent planners that do worse than simply using the commanded joints. The register, every stored row and the code that rebuilds each number are public.

Code and rows: `github.com/ravediamond/paper-register-reproducibility` **Reproduce:** `make check`. Every number carries its source file and protocol commit.

1 Introduction

A method that works in its paper is usually assumed to carry over, more or less intact, to a smaller rig. However, the 22 papers in this study were mostly shown on setups far from ours: 16 of them in simulation or on rigs above $20k, and 2 on a setup under $5k, which is where an SO-101 arm with a Jetson Orin NX 16 GB falls (the appendix gives the rig of every paper). It is therefore far from obvious which of their claims a small team can rely on, and which hold only under the conditions they were shown in.

In this work, we present a register of such re-tests, one row per claim, kept on a single €1,350 rig. It grew out of our own write-ups: each re-test was published with its data as it was run, and there are 29 so far, which this paper collects in one table. Consider one entry. An adaptive chunking paper claims that choosing the chunk length from the entropy of sampled chunks follows the phases of the task, long in transport and short at the grasp. On our edge GPU, however, a single such decision takes 4,751 ms against a budget of 298 ms, so the entry records a fail on this rig; it says nothing about the method on the hardware it was shown on. Next to the verdict, the entry keeps the claim in the paper's words, our pass mark, our measurement and the published rows.

Overall, 12 of the 37 claims we tested reproduce on this rig and 18 fail, for a total compute cost of $45.10. Notably, several of the failures are not near misses against the pass mark but losses to a baseline that requires no learning at all, which leads us to propose three classes of failure (Section 4).

2 How an entry is made

Every entry rests on a pass mark committed to version control before its test runs, so that a result cannot move the bar after the fact; Section 5 checks from the commit history how well we kept to this. Before running the learned model itself, we also evaluate the cheapest signal it has to beat on the same rows (copy-last for a predictor, the commanded joints for a learned latent, episode length for a judge). This costs minutes, and in several entries it is this baseline, rather than the pass mark, that the method fails against.

The papers were chosen for the stack this rig runs, namely world models, VLA policies, failure monitors, action chunking and edge latency, so the failure rate describes this register and not the field. To avoid choosing papers after seeing their results, the register file lists every paper we have tested and every paper scheduled next, each entered before its test.

3 Results

Figure 1 summarises the register, and Table 3 counts the verdicts both by paper and by claim row; the appendix gives every row with its claim, pass mark and measurement.

Finding. On claims taken from papers, 12 of 37 reproduce and 18 fail. The remaining 8 rows test our own baselines (e.g., that the commanded joints beat the world model), and 5 of those hold.

At the level of papers, 4 of the 22 reproduce and 13 fail on this rig.

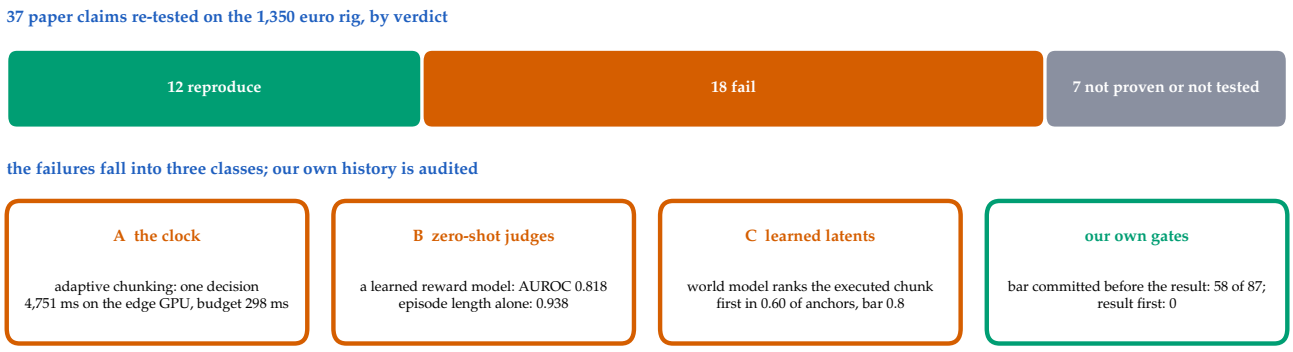


Figure 1: what survived when the papers left the lab: of the 37 paper claims, 12 reproduce and 18 fail; the failures fall into three classes (the clock, zero-shot judges against a trivial signal, learned latents against the commanded joints); our own gates have the bar committed before the result in 58 of 87 and the result first in 0.

Table 1: Verdicts by paper and by claim row (claim rows include our own baseline comparisons).

verdict	papers	claim rows
reproduces	4	17
fails	13	20
not proven	3	3
not tested	1	4
withdrawn	1	1
total	22	45

4 Three failure classes

The adaptive chunking entry of Section 1 misses the real-time budget on the edge GPU, although the method may well work on faster hardware; we call this kind of failure class A, the clock. In a second case, a model judged outcomes on this rig without any training on it. Used as a failure detector, a learned reward model reached an AUROC of 0.818, whereas episode length alone reaches 0.938. Class B covers such judges of success, failure or progress when they lose to a trivial signal. Class C concerns learned latents: a world model ranked the executed chunk first in 0.60 of anchors, against a pass mark of 0.8, while an action-only baseline reaches 0.68–0.90. Class C thus holds latents used to plan, rank or track progress that lose to the joints, to copy-last or to an action-only baseline.

These examples illustrate the classes rather than count them. The classes are a proposal, drawn from the author's reading of the 20 failed claim rows, and a second reader would show whether they hold up. In practice, they suggest an order of checks for anyone with a similar rig: time the method on the target device first, then compare it with episode length or the commanded joints on the same rows, and only then with its pass mark.

5 How far the register can be trusted

A verdict is only meaningful if its pass mark was fixed before its result. Notes and commit dates can be edited by hand; commit ancestry, in contrast, cannot be changed without rewriting history that others have already cloned. For each gate, we therefore find the first commit that contains its bar and the first that contains its result, and compare the two (`repro/prereg-check/scripts/prereg_audit.py`, abridged):

```
def ancestor(a, b):
    return subprocess.run(["git", "-C", str(RR), "merge-base", "--
      is-ancestor", a, b]).returncode == 0
...
            elif b[0] == res[0]:
                cls = "same-commit"
            elif ancestor(b[0], res[0]):
                cls = "bar-first"
            elif ancestor(res[0], b[0]):
                cls = "result-first"
```

Table 1 gives the resulting classes for the 87 gates that ran and were logged by 2026-10-31: most are bar-first and 0 are result-first. A same-commit gate is one whose bar and result entered the repository together, which says nothing about which was written first. A first pass by text search alone found 31 same-commit gates, mostly from batch logging; going back to the earlier files that hold their bars moved 21 of them to bar-first and 4 to no-bar. The audit table in the repository gives the evidence line for every move.

The early gates also mixed point estimates with different kinds of interval. We recomputed 10 of them from the stored rows with a single statistic, the episode bootstrap (2,000 resamples, percentile 95 % CI), and for 3 of them the bar falls inside the interval (Table 2). The false-intervention threshold was fitted to meet its rate, so its interval is descriptive only. The register keeps the pre-registered point verdicts and marks these three as fragile.

Finally, all 29 write-ups rebuild from their published rows, and the check for the cold reruns prints:

Table 2: Commit-order class of each gate. The audited column is the one the paper uses; the first pass is what text search alone gave on the same gates.

class	first text pass	audited, 2026-10-31	audited, today
bar-first	36	58	61
same-commit	31	6	7
result-first	—	0	0
no bar	19	23	23
unresolved	1	—	—
gates	87	87	91

Table 3: The three gates whose verdict flips somewhere inside its 95 % interval.

gate	n	estimate	95 % CI	bar	point verdict
colour judge (no learning), accuracy	87	94.2 %	89.6–98.9 %	$\geq$ 95 %	fail
oscillation among failed episodes	25	24.0 %	8.0–40.0 %	$\geq$ 10 %	pass
judge threshold, false interventions	42	2.4 %	0.0–7.1 %	$\leq$ 5 %	pass

```
$ python3 repro/cold-reproduction/l0_check.py
ok  posts: 29 (report 29)
ok  L0 PASS: 29 (report 29)
ok  L1 REPRO: 6 (report 6)
ok  L1 DIFF: 0 (report 0)
ok  L1 classes: {'REPRO': 6, -'': 3, 'SKIP': 8, 'FAIL-ENV': 12}
L0 PASS
```

We ran the cold reruns from a fresh clone with no credentials. 6 gates reran and matched, 5 of them exactly. A further 12 stop at the world-model environment, which is not yet published and whose release would unblock most of them; the remaining 11 need a larger GPU, a private checkpoint or the arm, or rely on a step the write-up leaves out. We also checked the citations and paper-side fields of the register against the papers' PDFs, and corrected them. The register names 38 arXiv ids; the ids missing from our mirror of the papers all belong to scheduled tests, and one tested paper has no arXiv id, so it could not be checked against a PDF.

6 Limitations

With a single operator, the author, running the rig and labelling the outcomes, observer bias is the main confound; labels were given with the arm at rest but not always blind to it. Where commit order shows it, the pass mark predates the run, so a label could not have moved the bar. For 29 of the 87 gates, however, the repository cannot show whether bar or result came first, and 21 of the bar-first verdicts rest on matching the bar text in an earlier file; a blind second reader would show whether any verdict flips. Beyond labelling, 3 verdicts are fragile under the bootstrap, and the cold reruns cover 6 gates so far, with the unpublished world-model environment blocking most of the rest.

7 Related work

Two works evaluate policies on the same arm. One is a standardized real-world benchmark that reports a taxonomy of failures and recoveries for VLA policies [13]; the other runs a fleet of SO-101 cells and evaluates policies at scale [14]. In simulation, the vla-eval tool runs VLAs across 14 benchmarks [8], and a benchmark of demonstration quality studies one method family [22]. In contrast to these, we do not rank policies: we test the claims behind methods (judges, world models, latency) against thresholds fixed before the run, on one inexpensive rig. The closest work is RoboSeed [27], which scores VLA and world-action models on five dimensions, including reproduction evidence and deployment affordability. Its reproduction-evidence score counts five audit milestones (configuration mapping, task records, task coverage, training records and seeds), and its authors state that the score is not a probability that a result is correct. Instead of such a score, we give one pass or fail per claim on a €1,350 rig, together with the rows behind it.

8 Conclusion

Conclusion. In this work, we presented a register of 37 claims from 22 robot-learning papers, re-tested on a €1,350 rig for $45.10 of compute, of which 12 held. Several of the failures were losses to a baseline that needs no learning, and every row behind the verdicts is published, so that any number in this paper can be rebuilt from the repository (`github.com/ravediamond/paper-register-reproducibility`). Our findings suggest that re-testing a paper on a cheap rig should start from its cheapest baseline on the same rows, with the threshold committed before the run; we plan to extend the register as the scheduled tests are run.

References

[1] DINO-WM: World Models on Pre-trained Visual Features enable Zero-shot Planning, 2024. arXiv:2411.04983.

[2] Can We Detect Failures Without Failure Data? Uncertainty-Aware Runtime Failure Detection for Imitation Learning Policies, 2025. arXiv:2503.08558.

[3] SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics, 2025. arXiv:2506.01844.

[4] Real-Time Execution of Action Chunking Flow Policies, 2025. arXiv:2506.07339.

[5] SAFE: Multitask Failure Detection for Vision-Language-Action Models, 2025. arXiv:2506.09937.

[6] VLA-JEPA: Enhancing Vision-Language-Action Model with Latent World Model, 2026. arXiv:2602.10098.

[7] Robometer: Scaling General-Purpose Robotic Reward Models via Trajectory Comparisons, 2026. arXiv:2603.02115.

[8] vla-eval: A Unified Evaluation Harness for Vision-Language-Action Models, 2026. arXiv:2603.13966.

[9] LeWorldModel: Stable End-to-End Joint-Embedding Predictive Architecture from Pixels, 2026. arXiv:2603.19312.

[10] Hierarchical Planning with Latent World Models, 2026. arXiv:2604.03208.

[11] Adaptive Action Chunking at Inference-time for Vision-Language-Action Models, 2026. arXiv:2604.04161.

[12] When Does LeJEPA Learn a World Model?, 2026. arXiv:2605.26379.

[13] Benchmarking Vision-Language-Action Models on SO-101: Failure and Recovery Analysis, 2026. arXiv:2606.08881.

[14] ArmnetBench v0.1: Parallel Real-World Evaluation of Manipulation Policies on a Low-Cost Arm Farm, 2026. arXiv:2607.24481.

[15] TurboVLA: Real-Time Vision-Language-Action Model at 32 Hz on an RTX 4090 with <1 GB VRAM, 2026. arXiv:2607.27205.

[16] ContactGuard: Pre-Contact Execution Monitoring with Action-Conditioned Latent World Models, 2026. arXiv:2608.13438.

[17] VLA-Corrector: Stage-Aware Observable State Understanding for Prompt-Based Closed-Loop Recovery of Vision-Language-Action Policies, 2026. arXiv:2609.06508.

[18] D-JEPA: A Decision-Aligned Latent World Model, 2026. arXiv:2609.24749.

[19] H-JEPA: End-to-End Learning of Hierarchical World Models for Visual Planning, 2026. arXiv:2610.06805.

[20] RoboJEPA: Scaling Robotic Latent World Models, 2026. arXiv:2610.10515.

[21] ReAP: From Visual Failure Recognition to Action-Conditioned Risk Prediction for Runtime Failure Monitoring, 2026. arXiv:2610.16309.

[22] What Matters in Quality-Aware VLA Pretraining: A Benchmark and Empirical Study, 2026. arXiv:2610.18869.

[23] REKI: Registering Knowledge in VLAs, 2026. arXiv:2610.22178.

[24] PACT-VLA: Speculative Action-Chunk Verification with Pre-Commit Observations, 2026. arXiv:2610.28019.

[25] Coupling Execution Horizon and Sampling Depth in Flow-Matching VLA Policies, 2026. arXiv:2610.32102.

[26] Tracking Task Progress Through Errors and Recovery:A Benchmark and Recurrent Distributional Model, 2026. arXiv:2610.39125.

[27] Xin Ma et al. RoboSeed: Multidimensional Evaluation of Vision-Language-Action and World-Action Models: Capability, Resource Affordability, and Research Evidence, 2026. arXiv:2610.40026.

Appendix: the register

paper	their rig	claim tested	bar	our result	verdict
LeWorldModel [9]	sim	A small JEPA world model learns the arm's dynamics well enough to plan with it.	Dynamics: beat copy-last, every corruption ≥ 1.3 at $K = 30$. Planning: rank the executed chunk first among 8 in ≥ 0.8 of anchors.	dynamics 0.31 vs 0.89 copy-last (reproduces); planning top-1 0.60 vs 0.68–0.90 for an action-only baseline (fail)	fail
DINO-WM [1]	sim	A predictor on frozen DINOv2 patches plans better than one on a learned latent.	G1b top-1 ≥ 0.8 at some target distance.	arm-pose R^2 0.92 (vs 0.57 for W1); G1b top-1 0.55–0.61; no-model baseline 0.68–0.90	fail
RoboJEPA [20]	>$20k	Pretrained RoboJEPA predicts our latents, uses the actions, and runs on the robot's computer.	9-step L1 ≥ 15 % below copy-last (300M) and ≥ 5 % below shuffled actions; 2-view encoder ≤ 200 ms on the Orin.	11.8 % below copy-last (300M; 1B 15.8 % on 30 clips); 9–10 % below shuffled; encoder 526 ms on the Orin NX	fail
H-JEPA [19]	sim	A slow second level makes latent distance track task progress.	Progress monotonicity Spearman $\rho \leq -0.80$; false stalls ≤ 10 %.	level-2 ρ -0.73; false stalls 52 %	fail
D-JEPA [18]	<$5k	A decision-aligned ranking operator on a frozen world model picks better action chunks.	G1b regret ≤ 0.10 and $\leq$ the action-only baseline + 0.05.	regret 0.263 (W1 alone 0.263; action-only -0.007)	not tested
ContactGuard [16]	$5–20k	An event-anchored probe on imagined latents predicts grasp failure better than the current latent.	AUROC ≥ 0.85 and ≥ 0.05 above the current latent.	0.70 imagined vs 0.82 current	fail
FAIL-Detect and SAFE [2] [5]	>$20k	A conformal band on a learned failure score keeps false alarms at α while catching failures.	Successes flagged $\leq 1/20$; failures flagged ≥ 50 %.	22.7 % of successes flagged at $\alpha = 5$ %; 46 % of failures caught	fail
ReAP [21]	$5–20k	A risk head on an action-conditioned latent warns before failures.	$\geq 5/8$ failures caught at $\leq 1/20$ false interventions; action-conditioned iff true $\geq$ shuffled + 0.05.	v1 AUROC 0.93 came from time and length; v2 0.789 vs 0.708 shuffled, 3/8 at 5/42	fail
VLA-Corrector [17]	sim	A verifier on observable history flags a failing rollout early enough to correct it.	$\geq 5/8$ failures at $\leq 1/20$ false interventions.	the paper's verifier family on our rollouts (a GRU on pooled RGB statistics and actions): 2/8 caught at 3/42 false interventions, AUROC 0.59; cosine drift of the W1 latent 0/8, AUROC 0.30	fail
Robometer [7]	>$20k	A 4B reward model judges SO-101 rollouts zero-shot.	Pooled AUROC ≥ 0.85 and within-policy ≥ 0.8 on ≥ 5 of 7 policies.	pooled AUROC 0.818 on 2099 ArmnetBench rollouts; episode length alone 0.938	fail
SmolVLA [3]	<$5k	A 450M VLA succeeds on an SO-101 task and can run on low-cost hardware.	10-trial operator-judged gate (no numeric pass mark registered); chunk inference within the async budget (667 ms).	9/10 on bowls; 974 ms in PyTorch, 238 ms with vla.cpp on the Orin NX	reproduces

Table 4: The register, one row per paper, generated from the register file (internal model and gate names kept). The rig price of €1,350 counts the parts with a stated price; 2 small parts are not priced yet. Read the verdict column against the rig each paper was shown on.

paper	their rig	claim tested	bar	our result	verdict
TurboVLA [15]	sim	Our claim that the architecture made the policy ignore its instruction.	Task swap $\geq 5°$ and per-layout $\geq$ 75 % $\rightarrow$ our port was the bug; $<$ 1° with the control $\geq 0.095 \rightarrow$ the data.	upstream's own trainer on our data: task swap 0.020°, per-layout 38.9 %; control, their released LIBERO checkpoint run here: 0.105 ($\approx 5.5°$ of pan)	withdrawn
VLA-JEPA [6]	sim	VLA-JEPA trains cheaply on SO-101 data and runs on the robot.	10-trial operator-judged gate (no numeric pass mark registered); for reranking (H0): oracle / mean error $\leq$ 0.7 and N = 8 $\leq 1.5\times$ the latency.	9/10 for $9.7; 355 ms per chunk on the Orin NX; reranking no-go (0.76, 1.84×)	reproduces
REKI [23]	sim	Policies without memory oscillate between poses at a measurable rate.	$\geq$ 10 % of failed episodes oscillate (their criterion).	20.3 % of all episodes, 24.0 % of failures; REKI trained on my data: 19/20 + 19/20 on the arm, oscillates in 1/40	reproduces
Real-Time Chunking [4]	unstated	Conditioning a new chunk on the executing one removes the jump at chunk boundaries.	Best gradient-free method cuts the median jump $\geq$ 50 % against naive async, drift $\leq$ +10 %, within 667 ms.	prefix overwrite cuts the jump 71–73 % against naive async at +0.4 ms; tuned LeRobot identity mode is close	not proven
HWM (hierarchical latent world models) [10]	>$20k	A high-level world model over learned macro-actions beats the low-level model beyond about 1 s (their Fig. 6 crossover).	Crossover in [1.0, 2.0] s; at 2 s high-level error $\leq$ 0.95 × low-level; learned macro-actions $\geq$ net joint displacement on cosine.	no crossover: W1 $0.382 \pm 0.005 \rightarrow 0.563 \pm 0.007$, high level $0.769 \pm 0.009 \rightarrow 0.759 \pm 0.009$ (0.73–2 s); cosine 0.608 ± 0.007 vs 0.551 ± 0.006	not proven
LeJEPA identifiability [12]	sim	A SIGReg JEPA trained on goal-directed data recovers the true latent state only partially and unevenly.	W1's mean R^2 from latent to joints < 0.80; anisotropy (max $-$ min per-joint R^2) ≥ 0.20; the higher-R^2 model has the higher G1b top-1.	W1 mean R^2 0.565, anisotropy 0.639 (shoulder lift 0.91, wrist flex 0.27); frozen DINOv2-S 0.867 / 0.301	reproduces
SURVEYOR	sim	A threshold on the imagined-vs-reached latent distance accepts good plans and rejects bad ones ($\tau = 0.20$).	τ below the 10th percentile of non-equivalent pairs and ≤ 0.40; as a monitor $\geq 5/8$ failures at $\leq 2/42$; $\geq$ 0.05 AUROC over a shuffled drafter.	$\tau = 1.249$ on W1 latents of 69 real rollouts; as a monitor 3/8 at 6/42 (S = 30), AUROC 0.756 vs shuffled 0.560	fail
Adaptive Action Chunking (AAC) [11]	>$20k	Chunk length chosen from the entropy of N sampled chunks follows the task's phases (long in transport, short at the grasp), lowers the boundary jump, and costs little with batched sampling.	median h* in transport $\geq$ grasp window + 2; jump with AAC + overwrite $\leq$ fixed + overwrite at d = 8; an N = 20 decision $\leq$ 298 ms on the Orin.	h* 2 in transport and 2 at the grasp (86 % of decisions at the floor); jump 0.973 vs 0.940; N = 20 decision 4,751 ms (sequential) on the Orin	fail
PACT-VLA [24]	$5–20k	Picking among N sampled chunks by their flow residual under the pre-commit observation fits what happens next better than a random or stale pick, with no added control-step delay.	PACT beats a random pick and the stale-observation residual at d = 8 (CIs < 0); its delay on the Orin $\leq$ the naive delay; at that delay it still beats a single sample.	offline $-0.27°$ vs random and vs the stale residual (both CIs < 0); on the Orin 1,200 ms per cycle, 37 control ticks vs 8	fail
Horizon × depth (Euler steps vs execution horizon) [25]	>$20k	With long open-loop commitment, cutting a flow-matching VLA's Euler steps from 10 to 2 contracts the action cloud around an unchanged centroid, saves most of the inference time and avoids edge-GPU thermal throttling.	median R(2)/R(10) $\leq$ 0.70 with the centroid within 0.05 relative and 3°; N = 2 latency $\leq 0.85 \times$ N = 10 on the Orin, N = 10 within 5 % of 238 ms.	R(2)/R(10) 0.78 (CI 0.73–0.81), centroid 0.018 and 2.35°; 196.1 vs 237.5 ms per chunk (0.83, vision 151 ms); no throttling at N = 10 on the Orin, at MAXN_SUPER or 25 W	not proven
REMORA [26]	>$20k	Reading a progress distribution out at the supported peak nearest the previous estimate steadies the estimate and lets it fall during real setbacks, with no worse detection of the stages.	Against per-frame argmax: premature not higher and missed $\leq$ +2 points (P1); flips halved, CI upper ≤ 0.7 (P2); a decrease inside $\geq$ 50 % of the reversal windows with $\leq$ 20 % false regressions (P3).	premature 0.22 vs 0.25, missed equal (P1 pass); flips 0.70 × argmax, CI 0.48–0.94 (P2 fail); setback recall 2/7, false 0.52 (P3 fail); MAE 0.184 vs median 0.218	fail

Table 4, continued.

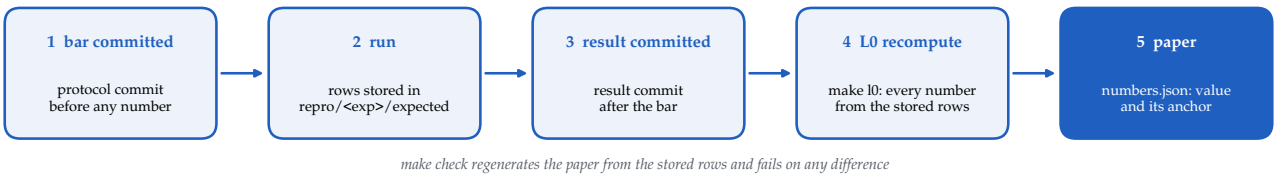


Figure 2: how every number in this paper is made: a protocol with its bar is committed before any number exists (1), the run stores its rows under repro/ (2), the result is committed after the bar and the commit order is audited (3), make l0 recomputes every number from the stored rows (4), make_paper.py writes numbers.json with each value's anchor and fills the text from it (5); make check regenerates the paper and fails on any difference.